1️⃣ Core Java & OOP – Interview Answers

1. What is a class and object?

- Class: Blueprint or template defining properties (fields) and behavior (methods).
- Object: Runtime instance of a class with actual values.

---

2. What are the main features of Java?

- Platform-independent, object-oriented, simple, secure, robust, multithreaded, and high-performance.
- Supports automatic memory management via garbage collection.

---

3. Explain platform independence.

- Java code is compiled into bytecode, which runs on any OS with a JVM.
- "Write Once, Run Anywhere" principle ensures cross-platform compatibility.

---

4. Why is Java not a pure object-oriented language?

- Supports primitive data types (int, char, boolean).
- Has static methods/variables that are not tied to objects.
- So, not 100% OOP.

---

5. Difference between JDK, JRE, and JVM

Component	Purpose	Includes
JVM	Executes bytecode	Runtime engine
JRE	Runs Java programs	JVM + Libraries
JDK	Develops Java programs	JRE + Compiler & tools

---

6. Explain the four pillars of OOP

- Encapsulation: Data hiding using private fields + getters/setters.
- Inheritance: Reuse properties/methods from parent class.
- Abstraction: Hide implementation details using abstract classes/interfaces.
- Polymorphism: Same method name behaves differently (overloading/overriding).

---

7. Polymorphism vs Method Overloading vs Method Overriding

- Polymorphism: "Many forms" of a method.
- Overloading: Same name, different parameters, compile-time.
- Overriding: Same name & parameters in subclass, runtime.

---

8. Final keyword usage

- final class: Cannot be inherited.
- final method: Cannot be overridden.
- final variable: Constant value.

---

9. Difference between == and .equals()

- == → checks reference equality.
- .equals() → checks content equality.

🔟 Stack vs Heap Memory

- Stack: Stores local variables & method calls; automatic memory.
- Heap: Stores objects; managed by garbage collector.

---

11. Static keyword

- Shared by all objects of a class.
- Used for class variables, methods, and blocks.
- Accessed without creating an object.

---

12. Interface vs Abstract class

- Interface: Only abstract methods (till Java 7), multiple inheritance, no constructors.
- Abstract class: Abstract + concrete methods, single inheritance, can have constructors.

---

13. hashCode() and equals() contract

- If two objects are equal, their hashCode() must be equal.
- equals() checks content equality, hashCode() used in hash-based collections.

---

14. HashMap vs ConcurrentHashMap

- HashMap: Not thread-safe, allows one null key, fast.
- ConcurrentHashMap: Thread-safe, segmented locking, no null keys.

---

15. Exception handling in Java

- Use try-catch-finally blocks.
- Checked exceptions → must handle (IOException), unchecked → runtime (NullPointerException).
- throw to manually throw, throws to declare.

---

16. Multithreading in Java

- Thread: Smallest execution unit.
- Create via Thread class or Runnable interface.
- Example using Runnable & ExecutorService:

```
Runnable task = () -> System.out.println("Hello Thread");
ExecutorService ex = Executors.newFixedThreadPool(2);
ex.submit(task);
ex.shutdown();
```

---

17. Synchronization & thread issues

- Synchronization: One thread accesses shared resource at a time.
- Issues: Deadlock (threads waiting), Starvation (low-priority thread not executing).

---

18. finalize() method

- Called by garbage collector before object destruction.
- Used to release resources like file or network connections.
- Not guaranteed to run immediately.

1️⃣ What is Java?

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems.
It is widely used for building desktop, web, and mobile applications.
Java follows the principle “Write Once, Run Anywhere (WORA).”

2️⃣ What are the main features of Java?

Java is simple, object-oriented, secure, portable, and platform independent.
It supports multithreading, automatic memory management, and exception handling.
It’s also robust and high-performance due to the JIT compiler.

3️⃣ Why is Java called platform independent?

Java programs are compiled into bytecode, not machine code.
This bytecode can run on any operating system with a JVM (Java Virtual Machine).
Thus, the same Java program can execute anywhere.

4️⃣ Why is Java not a pure object-oriented language?

Because it supports primitive data types like int, char, and boolean.
It also allows static methods and variables, which are not associated with objects.
Hence, it’s not 100% object-oriented.

5️⃣ What is bytecode?

Bytecode is an intermediate code generated after Java source code compilation.
It is a .class file that the JVM executes.
It makes Java portable and platform-independent.

6️⃣ What is JVM, JRE, and JDK?

- JVM (Java Virtual Machine): Executes bytecode at runtime.
- JRE (Java Runtime Environment): JVM + libraries needed to run Java apps.
- JDK (Java Development Kit): JRE + compiler and development tools.

7️⃣ What is the difference between JDK, JRE, and JVM?

- JDK: Used to develop Java programs.
- JRE: Used to run Java programs.
- JVM: Converts bytecode to machine code and executes it.

  👉 JDK > JRE > JVM (contains hierarchy).

8️⃣ Explain the Java compilation and execution process.

1️⃣ Java source code (.java) is compiled by javac into bytecode (.class).

2️⃣ The JVM loads this bytecode.

3️⃣ The JIT compiler translates it to machine code, and the program executes.

9️⃣ What are identifiers and keywords in Java?

- Identifiers: Names given to variables, classes, or methods (e.g., name, Student).
- Keywords: Reserved words with special meaning (e.g., class, int, if).
  Identifiers are user-defined; keywords are predefined.

🔟 What are data types in Java?

Java has primitive and non-primitive data types.
Primitive types: byte, short, int, long, float, double, char, boolean.
Non-primitive types: String, Arrays, Classes, Interfaces.

1️⃣1️⃣ What are literals in Java?

Literals are fixed constant values assigned to variables.
Examples: 10 (integer), 'A' (char), "Hello" (string), true (boolean).
They represent the actual value in the code.

---

1 2 What are type casting and widening/narrowing conversions?

Type casting is converting one data type to another.
Widening: smaller → larger type (int → long) — automatic.
Narrowing: larger → smaller type (double → int) — needs explicit cast.

---

1 3 What is the difference between primitive and non-primitive data types?

- Primitive: Predefined by Java, store single values (e.g., int, char).
- Non-primitive: Created by users, can store multiple values (e.g., String, Array).
  Primitive types store value directly; non-primitives store reference.

---

1 4 What are access modifiers in Java?

Access modifiers define the scope and visibility of classes, methods, and variables.
Types: public, protected, default, private.
They help in encapsulation and code security.

---

1 5 What is the default value of instance variables in Java?

Instance variables get default values if not initialized.
Examples: int → 0, boolean → false, object → null.
Local variables, however, must be initialized before use.

---

1 6 What is the difference between local, instance, and static variables?

- Local: Declared inside methods, accessible only within.
- Instance: Declared in class but outside methods, unique per object.
- Static: Shared by all objects, belongs to the class.

---

1 7 What is the role of the main() method in Java?

The main() method is the entry point of every Java program.
Syntax: public static void main(String[] args).
It's automatically called by the JVM to start execution.

---

1 8 Can we overload the main() method in Java?

Yes, we can overload main() by changing parameters.
But JVM always calls the version with String[] args.
Other overloaded versions must be called manually.

---

1 9 What is the difference between constructor and method?

- Constructor: Initializes objects, same name as class, no return type.

- Method: Defines behavior, can return values, any name.
  Constructors are called automatically when objects are created.

---

2 0 What are static blocks and instance blocks?

- Static block: Runs once when the class is loaded (used for static initialization).
- Instance block: Runs every time an object is created (before constructor).
  Both are used for initialization of variables.

⚙️ OOPs – Java Interview Questions and Answers (21–40)

---

2 1 What are the four main OOP principles?

The four pillars of OOP are Encapsulation, Inheritance, Polymorphism, and Abstraction.
They help in modular, reusable, and maintainable code.
OOP makes complex programs easier to manage.

---

2 2 Explain encapsulation with an example.

Encapsulation is binding data and methods together in one class.
It hides internal details using private variables and public getters/setters.
Example:
private int age; public void setAge(int a){ age = a; }

---

2 3 What is inheritance in Java?

Inheritance allows a class to inherit properties and methods from another class using the extends keyword.
It promotes code reusability and hierarchy.
Example: class Dog extends Animal.

---

2 4 What is the difference between single and multiple inheritance?

- Single Inheritance: One class inherits another (A → B).
- Multiple Inheritance: One class inherits from multiple classes — not supported in Java (to avoid ambiguity).
  But it can be achieved using interfaces.

---

2 5 What is method overloading?

Overloading means same method name with different parameters (type or number).
It happens at compile-time (static polymorphism).
Example: add(int a, int b) and add(double a, double b).

---

2 6 What is method overriding?

Overriding means redefining a method in a subclass that exists in the parent class.
It occurs at runtime (dynamic polymorphism).
The method signature must be exactly same.

---

2 7 What is the difference between overloading and overriding?

- Overloading: Compile-time, same name different parameters, same class.
- Overriding: Runtime, same name and parameters, different classes (inheritance).
  Overloading increases flexibility, overriding adds specialized behavior.

---

2 8 What is abstraction in Java?

Abstraction hides implementation details and shows only essential features.
It can be achieved using abstract classes and interfaces.
It helps reduce complexity.

---

2 9 What are abstract classes and abstract methods?

- Abstract class: Declared with abstract keyword, cannot be instantiated.
- Abstract method: Declared without a body, implemented by subclasses.
  Example:

abstract class Shape { abstract void draw(); }

---

3 0 What is an interface?

An interface is a collection of abstract methods and constants.
It provides complete abstraction and supports multiple inheritance.
Implemented using the implements keyword.

---

3 1 Difference between abstract class and interface?

- Abstract class: Can have abstract + concrete methods.
- Interface: Only abstract methods (till Java 7), supports multiple inheritance.
  Abstract class uses extends, interface uses implements.

---

3 2 Can an interface extend another interface?

Yes ✅ an interface can extend multiple interfaces using the extends keyword.
Example:
interface A {} interface B extends A {}
This enables multiple interface inheritance.

---

3 3️⃣ Can an abstract class implement an interface?

Yes, an abstract class can implement an interface partially.
It can leave some methods unimplemented for subclasses to complete.
This is common in framework design.

---

3 4️⃣ Can we create an object of an abstract class?

No ❌ abstract classes cannot be instantiated directly.
They are meant to be extended by subclasses.
However, we can create reference variables of abstract class type.

---

3 5️⃣ What is polymorphism in Java?

Polymorphism means one name, many forms.
It allows methods to behave differently based on the object.
Types: Compile-time (overloading) and Runtime (overriding).

---

3 6️⃣ What is dynamic method dispatch?

It's the process of calling overridden methods at runtime using a parent class reference.
The JVM determines which method to invoke based on the actual object.
This is the essence of runtime polymorphism.

---

3 7️⃣ What is the use of the final keyword in Java?

- final variable: value cannot be changed.
- final method: cannot be overridden.
- final class: cannot be inherited.
  Used to prevent modification.

---

3 8️⃣ What is the difference between this and super keyword?

- this: Refers to the current class instance.
- super: Refers to the parent class instance or constructor.
  Used to resolve variable/method conflicts.

---

3 9️⃣ Can a constructor be inherited?

No ❌ constructors are not inherited in Java.
However, the child class can call the parent constructor using super().
Each class defines its own constructor logic.

---

4 0 What is a static method and can it be overridden?
Static methods belong to the class, not the object.
They cannot be overridden, but can be redeclared (hidden) in a subclass.
Called using the class name, not object reference.

📝 Strings and Arrays – Java Interview Questions and Answers (41–57)

---

4 1 What is a String in Java?
A String is a sequence of characters enclosed in double quotes " ".
It is an immutable object of the java.lang.String class.
Example: String name = "Java";

---

4 2 What is the difference between String, StringBuilder, and StringBuffer?

- String: Immutable — can't be changed after creation.
- StringBuilder: Mutable, not thread-safe (faster).
- StringBuffer: Mutable, thread-safe (synchronized).

---

4 3 Why are Strings immutable in Java?
Strings are immutable for security, caching, and performance reasons.
It allows safe use in multi-threaded environments and String Pool.
Once created, a String's value cannot be modified.

---

4 4 How can you compare two Strings in Java?
Use equals() to compare content, and == to compare references.
Example:
s1.equals(s2); // true if same text
For lexicographic comparison, use compareTo().

---

4 5 What is the difference between == and equals() in Strings?

- == → compares memory addresses (references).
- equals() → compares actual string content.
  Hence, always use equals() for value comparison.

---

4 6 What are common String methods (length, trim, substring, split, indexOf, etc.)?

- length() → returns string length.
- trim() → removes spaces.
- substring() → extracts part of string.

- split() → breaks into words.
- indexOf() → finds character position.

---

4 7 How do you convert a String to a character array?

Use the toCharArray() method.

Example:

char[] ch = s.toCharArray();

Each character of the string is stored in an array.

---

4 8 How do you reverse a String in Java?

You can reverse using a loop, StringBuilder, or recursion.

Example:

new StringBuilder(s).reverse().toString();

Or manually swap characters from start to end.

---

4 9 What is an array?

An array is a collection of elements of the same data type.

It stores values in contiguous memory.

Example: int arr[] = {1, 2, 3, 4};

---

5 0 How do you declare and initialize arrays in Java?

Declaration: int[] arr;

Initialization: arr = new int[5]; or int[] arr = {1,2,3};

Arrays are objects stored in heap memory.

---

5 1 What is the difference between one-dimensional and multi-dimensional arrays?

- 1D array: Linear, like a list (int[] arr).
- 2D array: Matrix form (int[][] arr).
  2D arrays are arrays of arrays.

---

5 2 How to find the length of an array?

Use the length property (no parentheses).

Example:

int n = arr.length;

It returns the total number of elements.

---

5 3️⃣ Can we change the size of an array once created?

No ❌ arrays have fixed size once initialized.
To increase size, create a new array and copy elements.
Or use dynamic structures like ArrayList.

---

5 4️⃣ How to copy or clone an array?

Use one of the following methods:

- clone() → int[] b = a.clone();
- System.arraycopy(a, 0, b, 0, a.length);
- Arrays.copyOf(a, newLength);

---

5 5️⃣ How to find the largest/smallest element in an array?

Loop through all elements using a variable to track max/min.
Example:
if(arr[i] > max) max = arr[i];
Or use Arrays.stream(arr).max().getAsInt();

---

5 6️⃣ How to sort an array in Java?

Use the built-in method:
Arrays.sort(arr);
It sorts elements in ascending order using Dual-Pivot Quicksort.

---

5 7️⃣ What is the difference between Arrays and ArrayList?

- Array: Fixed size, can store primitives or objects.
- ArrayList: Dynamic size, only stores objects.
  ArrayList provides more flexibility and built-in methods.

⚠️ Exception Handling – Java Interview Questions and Answers (58–73)

---

5 8️⃣ What is an exception?

An exception is an unexpected event that disrupts normal program flow.
It occurs during runtime (like division by zero, file not found).
All exceptions are subclasses of the Throwable class.

---

5 9️⃣ What is the difference between error and exception?

- Error: Indicates serious system issues (like memory overflow), not recoverable.

- Exception: Problems in code logic, can be handled using try-catch.
  Errors are in java.lang.Error, exceptions in java.lang.Exception.

---

6 0 What are checked and unchecked exceptions?

- Checked: Checked at compile-time (e.g., IOException, SQLException).
- Unchecked: Checked at runtime (e.g., NullPointerException, ArithmeticException).
  Checked must be handled or declared using throws.

---

6 1 What is the use of try-catch block?

The try block contains code that may throw exceptions.
The catch block handles those exceptions gracefully.
It prevents program termination due to runtime errors.

---

6 2 What is the purpose of finally block?

The finally block executes always, whether exception occurs or not.
It's used for cleanup operations like closing files or connections.
Example:
finally { System.out.println("Closed resources"); }

---

6 3 Can a try block exist without a catch block?

Yes ✅, a try block can exist with a finally block instead of catch.
Either catch or finally is mandatory with try.
Example:
try { ... } finally { ... }

---

6 4 Can we have multiple catch blocks in Java?

Yes ✅, multiple catch blocks can handle different exception types.
They must be ordered from most specific to most general.
Example:
catch(IOException e) { } catch(Exception e) { }

---

6 5 What is the use of the throw keyword?

The throw keyword is used to manually throw an exception.
Example:
throw new ArithmeticException("Invalid division");
Used inside methods or blocks.

---

6 6 What is the use of the throws keyword?

The throws keyword declares potential exceptions a method might throw.

Example:

void readFile() throws IOException { ... }

It informs the caller to handle the exception.

---

6 7 What is the difference between throw and throws?

- throw: Used to explicitly throw one exception inside a method.
- throws: Used in method declaration to declare possible exceptions.
  Example:
  throw new IOException(); vs void m() throws IOException.

---

6 8 What are some common exceptions in Java?

- NullPointerException
- ArithmeticException
- ArrayIndexOutOfBoundsException
- NumberFormatException
- ClassNotFoundException
  These are part of java.lang package.

---

6 9 What is exception propagation?

When an exception is not handled in a method, it's passed up the call stack.

The JVM searches for a matching catch block in calling methods.

If none found, the program terminates abnormally.

---

7 0 What is custom exception?

A custom (user-defined) exception extends the Exception class.

It's used for application-specific error handling.

Example:

class InvalidAgeException extends Exception { ... }

---

7 1 What happens if an exception is not handled?

If not handled, the JVM terminates the program abruptly.

It prints the exception type, message, and stack trace.

Hence, handling ensures smooth program flow.

---

7 2️⃣ Can we write try-catch inside another try-catch?

Yes ✅, this is called nested try-catch.
It allows specific exception handling within different code blocks.
Useful for handling multiple risky operations separately.

---

7 3️⃣ What is the use of the Throwable class?

Throwable is the superclass of all errors and exceptions in Java.
It provides methods like getMessage(), printStackTrace(), and getCause().
Subclasses: Exception and Error.

📝 5. Multithreading (Basics)

---

7 4️⃣ What is a thread in Java?

A thread is the smallest unit of execution within a process.
It allows programs to perform multiple tasks concurrently.
Every Java application has at least one thread — the main thread.

---

7 5️⃣ What is multithreading?

Multithreading means executing multiple threads simultaneously.
It improves performance and CPU utilization.
Used in games, servers, and background operations.

---

7 6️⃣ What is the difference between process and thread?

- Process: Independent program with its own memory space.
- Thread: Sub-unit of a process sharing the same memory.
  Threads are lighter and faster than processes.

---

7 7️⃣ How to create a thread in Java? (Runnable vs Thread class)

1️⃣ Extend the Thread class and override run().
2️⃣ Implement the Runnable interface and pass it to a Thread object.

Example:

```
new Thread(new MyRunnable()).start();
```

---

7 8️⃣ What is the lifecycle of a thread?

States: New → Runnable → Running → Blocked/Waiting → Terminated.
Managed by the Thread Scheduler inside the JVM.
Each thread passes through these states during execution.

---

7 9 What is the difference between start() and run()?

- run() executes like a normal method (no new thread).
- start() actually creates a new thread and then calls run().
  Hence, always use start() to begin a thread.

---

8 0 What is thread synchronization?

Synchronization ensures only one thread accesses shared resources at a time.
It prevents race conditions and maintains data consistency.
Achieved using the synchronized keyword.

---

8 1 What is a synchronized method and block?

- Synchronized method: Locks the entire method for one thread.
- Synchronized block: Locks only a portion of code.
  Used to improve performance by limiting lock scope.

---

8 2 What is deadlock in Java?

Deadlock occurs when two or more threads wait for each other's locked resources indefinitely.
No thread can proceed, causing a program freeze.
Prevented by careful lock ordering and using try-lock.

---

8 3 What is inter-thread communication?

It allows threads to coordinate actions using wait(), notify(), and notifyAll().
Commonly used in producer-consumer problems.
These methods belong to the Object class.

---

8 4 What is the difference between sleep() and wait()?

- sleep() pauses the thread for a specific time (Thread class).
- wait() releases the lock and waits until notified (Object class).
  sleep() doesn't release the lock, wait() does.

---

8 5 What is ExecutorService?

ExecutorService is a thread pool manager from java.util.concurrent.
It efficiently manages multiple threads without manual creation.
Example:

```
ExecutorService ex = Executors.newFixedThreadPool(5);
```

---

8 6 What is the use of join() method?
join() makes one thread wait for another to finish.
Used when thread order of execution matters.
Example: t1.join(); // main waits for t1

---

8 7 What is thread priority?
Thread priority decides the order of thread execution by the scheduler.
Range: 1 (MIN) to 10 (MAX), default is 5 (NORM).
Higher priority increases execution chance, but not guaranteed.

---

8 8 What are daemon threads?
Daemon threads are background threads that support main threads.
They automatically terminate when all user threads finish.
Example: Garbage Collector thread.

🧮 6. Collections Framework

---

8 9 What is the Java Collections Framework (JCF)?
JCF is a set of classes and interfaces that store and manipulate groups of data.
It provides ready-made data structures like List, Set, Map, Queue.
Located in java.util package.

---

9 0 What is the difference between Collection and Collections?

- Collection: Interface that defines data group structure (List, Set).
- Collections: Utility class with static methods like sort(), max().
  Example: Collections.sort(list);

---

9 1 What are the main interfaces in the Collections Framework?
The main interfaces are Collection, List, Set, Queue, and Map.
Each defines specific data storage behavior.
For example, List allows duplicates; Set doesn't.

---

9 2 What is the difference between List, Set, and Map?

- List: Ordered, allows duplicates (ArrayList, LinkedList).
- Set: Unordered, unique elements (HashSet, TreeSet).
- Map: Key-value pairs (HashMap, TreeMap).

---

9 3 What is the difference between ArrayList and LinkedList?

- ArrayList: Uses dynamic array, faster for search, slower for insert/delete.
- LinkedList: Uses nodes, faster for insert/delete, slower for search.
  Both maintain insertion order.

---

9 4 What is the difference between HashSet and TreeSet?

- HashSet: Unordered, uses hash table, faster.
- TreeSet: Sorted order, uses Red-Black tree, slower.
  TreeSet implements NavigableSet.

---

9 5 What is the difference between HashMap and Hashtable?

- HashMap: Not synchronized, allows one null key.
- Hashtable: Synchronized, doesn't allow null keys or values.
  HashMap is preferred in modern Java.

---

9 6 What is the difference between HashMap and ConcurrentHashMap?

- HashMap: Not thread-safe; fails in concurrent access.
- ConcurrentHashMap: Thread-safe; divides map into segments for faster access.
  Used in multithreaded environments.

---

9 7 How does HashMap work internally?

HashMap uses hashing to store key-value pairs in buckets.
It computes hash code → bucket index → stores as a node.
Collisions are resolved using linked list or tree (since Java 8).

---

9 8 What is a LinkedHashMap?

It maintains insertion order of entries using a doubly linked list.
It extends HashMap and provides predictable iteration order.
Useful for caching (like LRU cache).

---

9 9 What is a TreeMap?

A TreeMap stores key-value pairs in sorted order (based on natural ordering or Comparator).
It implements NavigableMap and uses a Red-Black tree internally.
Doesn't allow null keys.

---

10 0 What is the difference between Comparable and Comparator?

- Comparable: Defines natural order (compareTo() inside class).
- Comparator: Defines custom order (compare() external class).
  Used for sorting custom objects.

---

10 1 What is the difference between Iterator and ListIterator?

- Iterator: Works on all collections, one-directional.
- ListIterator: Works only on Lists, bidirectional (has previous()).
  ListIterator also allows element modification.

---

10 2 What is the fail-fast and fail-safe iterator?

- Fail-fast: Throws ConcurrentModificationException when structure changes (e.g., ArrayList).
- Fail-safe: Works on a copy of collection (e.g., ConcurrentHashMap).
  Fail-safe does not throw exceptions during modification.

---

10 3 What is the difference between synchronized and concurrent collections?

- Synchronized: Slower, locks entire collection (e.g., Collections.synchronizedList()).
- Concurrent: Faster, uses fine-grained locking (e.g., ConcurrentHashMap).
  Concurrent collections are preferred in multithreading.

---

10 4 What are generics in Java Collections?

Generics allow type safety in collections.
Example: ArrayList<String> list = new ArrayList<>();
They prevent runtime ClassCastException.

---

10 5 How do you sort elements in a List?

Use Collections.sort(list) for natural order.
Or use list.sort(Comparator) for custom sorting.
Example:
Collections.sort(names, (a,b) -> a.length() - b.length());

---

10 6 What is the difference between Queue and Deque?

- Queue: Follows FIFO (First In First Out).
- Deque: Double-ended queue, allows insert/remove at both ends.
  Implemented by classes like LinkedList, ArrayDeque.

---

10 7 What is PriorityQueue in Java?

PriorityQueue orders elements based on priority, not insertion order.
It uses natural ordering or Comparator.
Useful for scheduling and task management.

---

10 8 What is the difference between Stack and Queue?

- Stack: LIFO (Last In First Out).
- Queue: FIFO (First In First Out).
  Stack uses push() and pop(), Queue uses offer() and poll().

---

10 9 What are common collection classes used in real projects?

- ArrayList → storing ordered data
- HashMap → key-value mapping
- HashSet → unique elements
- PriorityQueue → priority-based tasks
- ConcurrentHashMap → thread-safe map

---

11 0 What is the difference between HashSet and LinkedHashSet?

- HashSet: Unordered, faster for operations.
- LinkedHashSet: Maintains insertion order using linked list.
  Both don't allow duplicates.

🌱 Spring Boot & SQL Interview Questions

---

1 Spring Boot Basics

1. What is Spring Boot?

Spring Boot is a framework to build stand-alone, production-ready Spring applications.
It eliminates boilerplate code with auto-configuration and embedded servers like Tomcat.

2. What are the main features of Spring Boot?

- Auto-configuration
- Embedded servers (Tomcat/Jetty)
- Starter dependencies for easy setup
- Production-ready metrics & monitoring

3. What is the difference between Spring and Spring Boot?

- Spring: Full framework, requires XML/config setup.
- Spring Boot: Opinionated, auto-configuration, minimal setup.
  Boot is built on top of Spring for faster development.

4. What are starters in Spring Boot?
Starters are predefined dependency descriptors for specific functionality.
Example: spring-boot-starter-web includes Tomcat, Spring MVC, and Jackson.
They simplify Maven/Gradle dependency management.
5. What is Spring Boot auto-configuration?
It automatically configures beans based on the classpath and defined properties.
Reduces manual bean setup.
Example: DataSource auto-configured if JDBC driver is present.
6. What is Spring Boot actuator?
Provides production-ready features like health checks, metrics, and monitoring.
Endpoints like /actuator/health and /actuator/metrics help in app management.
7. What is Spring Boot CLI?
Command-line tool to run Groovy scripts as Spring Boot applications.
Speeds up prototyping without building full Maven/Gradle projects.

2️⃣ Spring Boot Data & JPA

8. What is Spring Data JPA?
Spring Data JPA simplifies database access using JPA.
It reduces boilerplate DAO/repository code.
Supports CRUD operations, pagination, and custom queries.
9. How do you define a repository in Spring Boot?
Use @Repository interface extending JpaRepository or CrudRepository.
Example:
public interface UserRepository extends JpaRepository<User, Long> {}
10. What is the difference between CrudRepository and JpaRepository?

- CrudRepository: Basic CRUD methods (save, delete, findAll).
- JpaRepository: Extends CrudRepository + pagination, sorting, batch operations.

11. How do you define an entity in Spring Boot?
Use @Entity annotation on a class.
Map fields to table columns using @Id, @Column.
Example:
@Entity class User { @Id Long id; String name; }
12. What is the difference between @Component, @Service, @Repository, @Controller?

- @Component: Generic bean
- @Service: Business logic layer
- @Repository: DAO / database layer, handles exceptions
- @Controller/@RestController: Web layer

13. What is @SpringBootApplication?

Combination of:

- @Configuration (bean configs)
- @EnableAutoConfiguration (auto config)
- @ComponentScan (scan packages for components)

14. What is the difference between @RestController and @Controller?

- @Controller: Returns views (like JSP/HTML).
- @RestController: Returns data (JSON/XML) directly using @ResponseBody.

3️⃣ SQL / Database Related

15. What databases are commonly used with Spring Boot?

- MySQL, PostgreSQL, H2 (in-memory), Oracle, SQL Server.
  H2 is commonly used for testing.

16. What is H2 database?

H2 is an in-memory lightweight SQL database.

Used for development/testing without installing a full DB server.

17. How do you configure a database in Spring Boot?

Define properties in application.properties or application.yml:

spring.datasource.url=jdbc:mysql://localhost:3306/db

spring.datasource.username=root

spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update

18. What is the difference between @Query and method naming convention in Spring Data JPA?

- Method naming: Auto-generates queries (findByNameAndAge).
- @Query: Custom JPQL or native SQL queries.

19. What are JPQL and native queries?

- JPQL: Java Persistence Query Language; object-oriented.
- Native SQL: Direct SQL query executed in the database.

20. What is the difference between fetch types: EAGER and LAZY?

- EAGER: Loads related entities immediately.
- LAZY: Loads related entities only when accessed.
  Used in @OneToMany, @ManyToOne relationships.

21. What are transactions in Spring Boot?

Transactions ensure atomicity, consistency, isolation, durability (ACID).

Use @Transactional to mark methods as transactional.

If an exception occurs, changes are rolled back.

22. How do you implement pagination in Spring Boot JPA?

Use Pageable and Page<T> interface.

Example: findAll(Pageable pageable) returns a paginated result.

23. What is the difference between save() and saveAndFlush()?

- save(): Persists entity in persistence context, may delay commit.
- saveAndFlush(): Persists and immediately flushes changes to database.

24. How do you handle database exceptions in Spring Boot?

Use @Repository to automatically translate exceptions to DataAccessException.

Can also use try-catch or @ControllerAdvice for global handling.